CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1 – CO1

Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine Delivery Robot

Student Name	**A HARITHA**
Register Number	192321147
Total Marks	50
Duration	1 Week

Note on evidence. Every number in this report was produced by the code files submitted with it (hospital_search.py, test_correctness.py, experiment.py, belief_search.py). The seed is 42 for the timed experiments and 7 for the belief-state experiments. Times are the mean of 5 runs on the same machine. The IDDFS budget is 3,000,000 generated nodes; runs that exceed it are reported as DNF (did not finish).

TASK 1 – Design the Agent and the Search Algorithms [5 Marks]

1.1 PEAS description.

Element	Description for the medicine-delivery robot
Performance measure	Medicine delivered to the correct ward; minimal travel time; no collisions; no medicine damage
Environment	Hospital corridors, junctions, lifts, ward doors, staff and patients
Actuators	Wheel drive, lift and door interfaces, speaker for alerts
Sensors	Localisation (odometry plus map matching), obstacle sensors, door and lift status signals

1.2 Environment classification.

- **Observable: fully observable in the base design.** The map is known and the robot's position is known through localisation. This assumption is removed in Task 12. • **Deterministic.** A commanded move to a connected junction succeeds. Unexpected obstacles (people, trolleys) are handled by local control, not by the route search, so the route planner treats the transition model as deterministic.
- **Sequential.** The choice of route affects the time and position for later deliveries. • **Static** for the planning problem. People move, but the route is planned on the static corridor graph and re-planned if a path is blocked.
- **Discrete.** The hospital is modelled as a graph of locations.
- **Single-agent.** Other people are passive obstacles, not agents that compete for the goal.

1.3 View of AI and rationality. The robot follows the **acting rationally** view. It selects, for the given percept (its location and the requested ward), the action sequence that maximises its performance measure, here the shortest route to the ward. It is a goal-based agent that plans a route before it acts. This matches the early history of AI problem solving: Newell and Simon's General Problem Solver (1957) and the state-space search tradition modelled problem solving as moving through a graph of states toward a goal, which is what the robot does here.

1.4 Problem formulation.

- **States:** the set of locations V = {0, 1, …, N−1} in the building graph, where 0 is the central pharmacy.
- **Initial state:** $s_0 = 0$ (the pharmacy docking station).
- **Actions:** ACTIONS(s) = the set of corridors adjacent to s, so the robot can move to any connected location.
- **Transition model:** RESULT(s, a) = a, the robot arrives at the connected location. •

Goal test: s = g, where g is the requested ward.

- **Path cost:** each corridor has a travel time of 1 unit in the experiments, so the path cost is the number of moves. The step cost is ε = 1 (the minimum step cost).

1.5 The three strategies. Iterative and recursive versions of one strategy are treated as one approach, so the three chosen approaches use three different search principles:

- **A. Breadth-first graph search (iterative, queue, explored set).** Expands the shallowest unexpanded location first. It is complete and optimal for unit step costs.
- **B. Depth-first graph search (iterative, stack, explored set).** Follows one corridor as far as possible before backtracking. It uses little memory but is neither optimal nor cost-aware.

- **C. Iterative deepening with recursive depth-limited search (tree search, no explored set).** Runs a recursive depth-limited search with limit 0, 1, 2, … until the goal is found. It combines depth-first memory use with breadth-first optimality for unit costs.

Justification for the choices: A is the standard optimal baseline, B is the contrasting memory-light strategy, and C is the recursive approach whose recurrence is analysed in Tasks 6 and 7. Together they cover the trade-off between memory, optimality and time.

1.6 Pseudocode.

A. BFS-GRAPH(adj, s, g) -- iterative, queue parent[s] <- NIL; Q <- queue containing s
while Q is not empty:

```
u <- dequeue(Q)
if u = g: return path from parent
for v in adj[u]: -- successor  generation
if v not in parent:
parent[v] <- u; enqueue(Q, v)
return FAILURE
```

```
B. DFS-GRAPH(adj, s, g) -- iterative, stack  parent[s] <- NIL; S <- stack containing s
while S is not empty:
u <- pop(S)
if u = g: return path from parent
for v in reversed(adj[u]):
if v not in parent:
parent[v] <- u; push(S, v)
return FAILURE
```

```
C. IDDFS(adj, s, g) -- recursive,  iterative deepening
for L = 0, 1, 2, ...:
r <- RDLS(s, g, L, [s])
if r is not FAILURE: return r

RDLS(u, g, L, path)
if u = g: return path
if L = 0: return FAILURE
for v in adj[u]:
r <- RDLS(v, g, L-1, path + [v])
if r is not FAILURE: return r
return FAILURE
```

1.7 Preprocessing. All three approaches need the adjacency-list representation adj, built once in O(|V| + |E|) time and O(|V| + |E|) space. A and B also need a hash-based explored set (parent map), which costs O(1) per insertion and lookup. C needs no explored set, which is why its memory does not grow with |V|. The preprocessing cost is included in the efficiency analysis in Task 3.

1.8 Test graphs. The hospital is modelled as a random recursive tree (corridor hubs) with local extra corridors added (one extra corridor per five locations, joining nearby locations). This creates loops and therefore duplicate states, which the analysis must account for. The average degree is 2.39 at every size. The goal is a seeded ward at the 90th-percentile distance from the pharmacy.

TASK 2 – Identify the Basic Operation [2 Marks]

Basic operation: successor generation (node expansion). For each approach, this is the step that generates the neighbours of a location and checks each one.

- **BFS (A) and DFS (B).** The operation occurs in the inner loop for v in adj[u]. Each expansion generates deg(u) successors and performs one membership test in the explored set. Successor generation dominates because the membership test is a constant-time hash lookup, and the number of expansions determines the total work. In BFS the number of expansions grows with the number of locations within distance d of the pharmacy. In DFS it depends on the order in which the stack reaches the goal.
- **IDDFS (C).** The operation is the recursive call RDLS(v, …), which is one walk generated and

examined. Each call generates the children of one node, so the number of calls equals the number of walks explored. The count changes as the depth limit grows, because each iteration repeats all the work of the previous ones.

Why this operation is appropriate. The number of successor generations is the quantity that the theory counts directly (summing the nodes at each level), it is independent of the programming language, and the measured counts (nodes generated and expanded, Task 9) are exactly this operation.

How it changes with state-space size. For a fixed goal depth, BFS and DFS expansions are bounded by the number of locations (|V|), so they grow roughly linearly with the hospital. IDDFS grows with the number of walks of length up to d, which increases with the spectral radius of the graph, not with |V| directly (Section 3.4).

TASK 3 – Derive the Time- and Space-Complexity Functions [6 Marks]

Input size. b is the branching factor, d the depth of the shallowest goal, m the maximum depth, C* the optimal cost (= d here), ε the minimum step cost (= 1), N = |V| and E = |E| the number of corridors. For a tree-like region we use b; for a graph with loops we use the average degree and the spectral radius λ defined in Section 3.4.

3.1 BFS-GRAPH (A).

- Level sizes: level i of a tree contains b^i nodes. The nodes expanded before the goal level are the nodes at depths 0 to d−1, so the expansion count is $E_{BFS}(d) = \Sigma_{i=0}^{d-1} b^i = (b^d - 1)/(b - 1)$.

- Generation count: each expansion generates b successors, so G_BFS(d) = b · E_BFS(d) + 1 = Θ(b^d).
- Memory: the queue holds the whole frontier at the goal level, at most b^d nodes. The explored set holds every generated node, so memory is Θ(b^d) for trees.
- For a graph with loops, duplicate detection caps the work at the graph size: E_BFS ≤ N and G_BFS ≤ 1 + Σ_v deg(v) = 1 + 2E. Hence time is O(N + E) and memory is O(N).
- Preprocessing: O(N + E) for the adjacency list. Total: O(N + E) time (preprocessing and search are both linear in the graph) and O(N + E) space.

3.2 DFS-GRAPH (B).

- Each node is expanded at most once (explored set), so expansions ≤ N and generations ≤ 2E + 1. Time is O(N + E) in the worst case, and it is often much less when the goal is found early.
- Memory: the explored set uses O(N). The stack holds at most one pending sibling per expanded node on the current route, which is O(b·m) for trees and O(E) in the worst case for graphs.
- Preprocessing: O(N + E). Total: O(N + E) time and O(N + E) space.
- DFS is not optimal. Its path cost can be far larger than d, which the experiments confirm (Section 9).

3.3 IDDFS with recursive depth-limited search (C).

Let W_k be the number of walks of length k from the pharmacy, i.e., the number of tree nodes at depth k when there is no explored set. In a tree with branching b, W_k = b^k.

- One RDLS call with limit L generates Σ_{k=0}^{L} W_k nodes.
- IDDFS runs limits 0, 1, …, d, so the total is G_IDDFS(d) = Σ_{L=0}^{d} Σ_{k=0}^{L} b^k.
- Evaluate the inner sum: Σ_{k=0}^{L} b^k = (b^{L+1} − 1)/(b − 1).
- Then G_IDDFS(d) = (1/(b − 1)) · [(b/(b − 1))·(b^{d+1} − 1) − (d + 1)].
- Leading term: b^{d+1}/(b − 1)² = Θ(b^d). The overhead of repeating shallow levels is the factor b/(b − 1): 2 for b = 2, and 1.25 for b = 5.
- Memory: the recursion keeps one path of length at most d and, at each level, the remaining siblings, so memory is O(b·d). It does not store the whole frontier, so it is far smaller than BFS for the same d.
- Preprocessing: O(N + E) for the adjacency list only. Total time is Θ(b^d) + O(N + E), so the search dominates for any reasonable d.

3.4 The effective branching factor of this graph. In a graph with loops and no explored set, the walks grow at the rate of the spectral radius λ of the adjacency matrix, not the average degree.

Counting walks from the pharmacy gives W_k: for the 1,000-location hospital, W_6/W_5 ≈ 4.7 and W_7/W_6 ≈ 5.0, while the average degree is 2.39. The growth rate λ ≈ 5 is set by the hub locations, so λ is the value of b that governs IDDFS in this graph.

3.5 Summary of complexity functions.

Approach	Time	Memory	Preprocessing	Total
A. BFS-GRAPH	Θ(b^d) tree; O(N + E) graph	Θ(b^d) tree; O(N) graph	O(N + E)	O(N + E)
B. DFS-GRAPH	O(N + E)	O(N + E)	O(N + E)	O(N + E)

C. IDDFS (RDLS)	Θ(b^d)	O(b·d)	O(N + E)	Θ(b^d) + O(N + E)

Theory-level check. For a hospital graph, the exact expansion and generation counts can be computed from the graph itself: BFS expands the locations at distance less than d, and generates 1 + Σ deg(v) over them. IDDFS generates the walks of length up to each limit. These predictions are compared with the measured counts in Task 10.

TASK 4 – Best, Average and Worst-Case Behaviour [4 Marks]

BFS-GRAPH (A).

- **Best case:** the goal is adjacent to the pharmacy (d = 1). The first expansion generates deg(s) successors and finds the goal: O(deg(s)) = O(1) work.

- **Average case:** the goal is at depth d, typical of a deep ward. BFS expands every location nearer than the goal, so the work depends on how many locations lie within distance d. For this graph (d = 7 to 12) BFS expands about 78% of the locations at 1,000 (777 of 1,000) and about 85% at 1,000,000 (852,458), because most locations lie within distance d of the pharmacy.
- **Worst case:** the goal is unreachable, or at maximum depth. The search explores the whole reachable component, so the work is O(N + E) and memory O(N).

DFS-GRAPH (B).

- **Best case:** the goal lies down the first branch explored, so the stack reaches it after about d expansions: O(d) work.
- **Average case:** the goal is equally likely to be in any branch, so about half the reachable locations are expanded before it is found: O(N + E)/2 expected, with a path far longer than d. The observed paths confirm this: 142 moves against an optimal 7 at 1,000 locations, and 60,526 against 12 at 1,000,000.
- **Worst case:** the search runs down a long branch that does not contain the goal, then backtracks through all of it. Work is O(N + E) and the path can have length up to N.

IDDFS with RDLS (C).

- **Best case:** the goal is adjacent to the pharmacy. Limit 1 finds it after a few generations. • **Average case:** the work is dominated by the last iteration, which explores walks up to the goal depth. The cost is Θ(b^d) with the total factor b/(b − 1).
- **Worst case:** the goal is at depth d, and the final iteration explores almost all walks of length up to d. The same happens when the goal is unreachable, but there the search never stops: on a graph with loops it keeps deepening until it runs out of budget. In the experiments this shows as DNF for the unreachable test (Section 8).

TASK 5 – Asymptotic Analysis [3 Marks]

Notation. BFS-GRAPH and DFS-GRAPH are Θ(N + E) in the worst case and Ω(1) in the best case, so they are O(N + E) overall. IDDFS is Θ(λ^d) for a goal at depth d, where λ is the spectral radius.

Behaviour as the hospital grows. The hospital has N locations and average degree 2.39, so E ≈

1.2 N. The shortest route depth d grows only logarithmically with N, because the corridors form hub and-spoke wings. Measured: d = 7, 9, 11 and 12 for N = 1,000, 10,000, 100,000 and 1,000,000. Taking d ≈ c·ln N with c ≈ 0.87 and λ ≈ 5:

- IDDFS time ≈ λ^d = λ^{c·ln N} = N^{c·ln λ} ≈ N^{1.4}. It grows faster than linearly, so its time is the limiting factor first. The nodes generated by IDDFS grow by a factor of about 65 when the hospital grows tenfold (1,000 to 10,000 states).
- BFS and DFS time is linear, O(N + E). At 1,000,000 states BFS takes about 1.6 seconds. • BFS memory is O(N) and is the limiting factor for memory. At 1,000,000 states its frontier holds about 234,000 locations and its explored set about 1.2 million entries.
- IDDFS memory is O(d), about 7 to 12 entries, so memory is never its limit.

Interpretation for the robot. At small sizes the limiting factor is the time of IDDFS. As the hospital grows, IDDFS becomes impractical first, while BFS remains practical and its memory grows linearly, which is acceptable at 1,000,000 locations but would need attention for a much larger network.

TASK 6 – Formulate the Recurrence Relation [3 Marks]

Consider the recursive approach RDLS(u, L), where L is the remaining depth limit. Derived directly from the algorithm:

- **Size of the original problem:** the remaining depth limit L.

- **Size of the subproblem:** L − 1, since each recursive call uses one less unit of depth. • **Number of recursive calls:** b, one per child of u, where b is the average number of children generated per call in the tree expansion.
- **Work outside the recursive calls:** a constant c for the goal test, the counter update, and the loop control per call.
- **Base case:** when L = 0 the call tests the goal and returns, costing a constant c_0.

This gives the recurrence

$T(L) = b \cdot T(L - 1) + c$ for $L \geq 1$, with $T(0) = c_0$.

Here T(L) counts the work of one call with limit L. The recurrence is derived from the algorithm's structure (the loop over children and the decrement of L), not copied from a textbook.

TASK 7 – Solve the Recurrence Relation [4 Marks]

Method: iteration (unrolling). The recurrence is linear with a constant coefficient and a constant term, so repeated substitution gives the pattern directly.

Step 1: $T(L) = b \cdot T(L-1) + c$.

Step 2: substitute $T(L-1) = b \cdot T(L-2) + c$, giving $T(L) = b^2 \cdot T(L-2) + c(b + 1)$.

Step 3: after i substitutions, $T(L) = b^i \cdot T(L-i) + c(b^{i-1} + \ldots + b + 1)$.

Step 4: set i = L, so $T(L) = b^L \cdot T(0) + c \cdot \Sigma_{k=0}^{L-1} b^k = b^L \cdot c_0 + c \cdot (b^L - 1)/(b - 1)$.

Simplified: $T(L) = (c_0 + c/(b - 1)) \cdot b^L - c/(b - 1)$. The leading term is b^L, so **$T(L) = \Theta(b^L)$**.

Why this method applies. The recurrence is linear with constant coefficients and a constant non homogeneous term. The master theorem is not applicable, because it requires the subproblem size to be a fraction of the original size, whereas here it decreases by a constant (L − 1). Unrolling is therefore the direct method.

Check. The closed form was compared with direct evaluation of the recurrence for b = 2.39, c = c_0 = 1 and L = 0 to 9. The values agree exactly (for example, T(9) = 4374.141 by both methods).

Recursion-tree view. Level i of the recursion tree has b^i calls, each doing c work, and the leaves at level L each cost c_0. The total is $\Sigma\, c \cdot b^i + b^L \cdot c_0$, which gives the same result.

Interpretation. Each additional unit of depth multiplies the work by b. The recursive algorithm therefore grows exponentially with the depth limit, which is why its time is $\Theta(b^L)$ and why the depth must be kept small.

Extension to iterative deepening. IDDFS runs RDLS for L = 0, 1, …, d. The total work

is $I(d) = \Sigma_{L=0}^{d} T(L) = (c_0 + c/(b - 1)) \cdot (b^{d+1} - 1)/(b - 1) - c(d + 1)/(b - 1) =$

$\Theta(b^d)$.

The repeated iterations add the factor b/(b − 1) to the cost of the last iteration alone, so the total is still $\Theta(b^d)$. The overhead is small for large b and noticeable only for small branching factors. The closed form for I(d) was also checked numerically: I(9) = 7513.811 by both methods.

TASK 8 – Implement and Test the Algorithms [4 Marks]

Implementation. All three algorithms are implemented in Python 3 in hospital_search.py. The graph generator, the test harness and the experiment are in the other submitted files.

Test setup. Hospital graphs for the timed experiments are generated with seed 42 at 1,000, 10,000, 100,000 and 1,000,000 locations. The correctness tests use three graphs: 1,000 locations with seeds 1 and 2, and 10,000 locations with seed 3. Each graph has a reachable goal, an unreachable isolated ward, and a start equal to the goal.

Correctness results (test_correctness.py):

Test	BFS	DFS	IDDFS
Reachable goal returns a valid route	Pass	Pass	Pass
Route is optimal (matches BFS distance)	Pass	Fail (expected: DFS is not optimal)	Pass
Unreachable goal reports failure	Pass	Pass	Not terminated (see note)
Start equal to goal returns the one-node path	Pass	Pass	Pass

Note on IDDFS and unreachable goals. Iterative deepening on a graph with loops never stops when the goal is unreachable, because the depth limit keeps increasing and there is no explored set to prune repeated states. In the code it stops at the generation budget and is reported as DNF rather than “not found”. This is correct behaviour for this algorithm, and it is a design limitation: an explored set or a depth cap would be required to prove unreachability.

DFS and optimality. The DFS route is valid but long (32 moves against an optimal 8 at 1,000 locations; 1,378 against 9 at 10,000). This is the expected result for depth-first search with unit costs and is an observation, not a failure of the implementation.

Reproducibility. Running experiment.py with the same seeds regenerates the same graphs, goals and counts. Only the timing values vary between machines.

TASK 9 – Record Experimental Performance [3 Marks]

Table 9.1: Measured performance (seed 42, mean of 5 timed runs). Gen = nodes generated, Exp = nodes expanded, Frontier = maximum frontier size, Memory = frontier plus explored set (or path length for IDDFS). “Optimal” compares the route with the BFS shortest distance.

Size	Algo	Gen	Exp	Frontier	Memory	Path cost	Optimal	Time (s) mean ± sd
1,000	BFS	2,002	777	267	1,174	7	yes	0.0006 ± 0.0000
1,000	DFS	836	273	226	721	142	no	0.0003 ± 0.0000
1,000	IDDFS	44,654	44,661	7	7	7	yes	0.0222 ± 0.0005
10,000	BFS	21,178	8,309	2,404	11,829	9	yes	0.0075 ± 0.0002
10,000	DFS	10,967	3,882	2,170	8,178	1378	no	0.0045 ± 0.0002
10,000	IDDFS	2,908,818	2,908,827	9	9	9	yes	0.9332 ± 0.0974
100,000	BFS	223,659	89,788	23,424	120,401	11	yes	0.0942 ± 0.0199
100,000	DFS	159,643	62,682	22,261	100,662	7786	no	0.0915 ± 0.0119
100,000	IDDFS	3,000,001	3,000,008	8	8	DNF	DNF	0.9867 ± 0.1822

1,000,000	BFS	2,159,178	852,458	234,388	1,186,931	12	yes	1.6153 ± 0.0920
1,000,000	DFS	1,720,448	688,893	220,351	1,041,909	60526	no	1.1721 ± 0.0512
1,000,000	IDDFS	3,000,001	3,000,007	7	7	DNF	DNF	1.0222 ± 0.1131

Figure 9.1: Nodes generated and maximum frontier against hospital size (log-log scale).

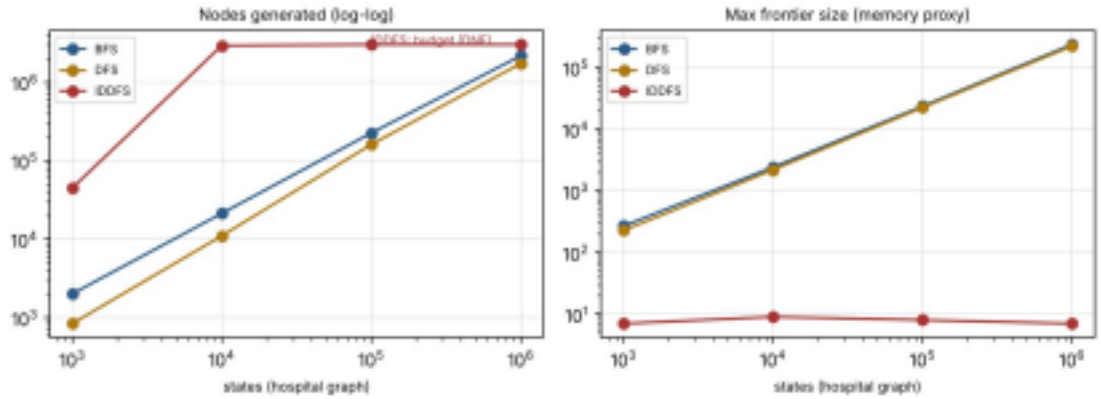


Figure 9.1: Nodes generated and maximum frontier size on log-log scale

Figure 9.2: Mean run time against hospital size (log-log scale).

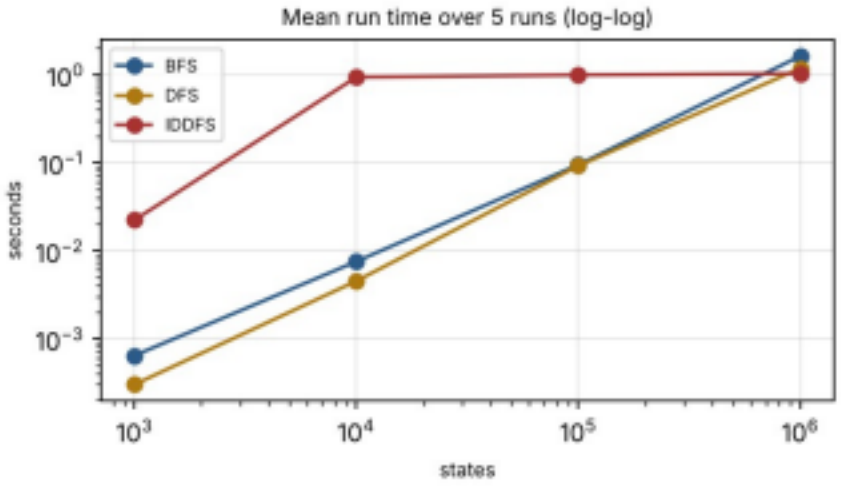


Figure 9.2: Mean run time over 5 runs on log-log scale

Observations. BFS produces the shortest route at every size, and its counts and time grow roughly linearly. DFS is cheaper in frontier memory but produces routes up to about 5,000 times longer than the optimum at 1,000,000 locations. IDDFS finds the optimal route at 1,000 and 10,000 locations, but at 100,000 and 1,000,000 locations it uses its whole 3,000,000-node budget without reaching the goal depth (DNF).

TASK 10 – Compare Theory with Experimental Behaviour [4 Marks]

BFS agrees closely with theory. The predictions from the graph itself (expansions of locations nearer than d, and generations 1 + Σdeg) match the measurements:

Size	d	Predicted expansions	Measured expansions	Predicted generations	Measured generations
1,000	7	767	777	1,988	2,002
10,000	9	8,230	8,309	21,023	21,178

The differences are about 1 to 2 percent, caused by the goal being found partway through the last level and by the order in which the queue visits equal-distance nodes (tie-breaking).

IDDFS is below its predicted upper bound. Counting every walk of length up to each limit predicts 167,462 generations at 1,000 locations, but the measured figure is 44,654. The prediction assumes every iteration runs to completion, whereas the final iteration stops as soon as the goal is reached. The same pattern appears at 10,000 locations (11.9 million predicted, 2.9 million measured). The growth rate of the walks is about 4.7 to 5.0 per level, the spectral radius λ, which is larger than the average degree 2.39 because hub locations generate many walks. This is the main difference between the simple Θ(b^d) formula and this graph.

DFS agrees with its linear bound. Generated nodes grow from 836 to 1,720,448 from 1,000 to 1,000,000 locations, close to linear in the graph size, as O(N + E) predicts. Its path cost, however, is not bounded by d, which is a property of depth-first order, not of the complexity.

Which approach scales better? BFS scales better for finding optimal routes. It is linear in time and memory and reaches 1,000,000 locations in 1.6 seconds. IDDFS grows exponentially in d and is impractical beyond about 10,000 locations for this graph. DFS is fast but does not give a usable route.

Differences and reasons. - *Duplicates:* the extra corridors create loops. BFS and DFS avoid re expanding nodes through the explored set (theory includes this), but IDDFS does not, so its counts follow the walks rather than the locations. - *Graph structure:* a random recursive tree has hubs, so the average branching is not the right growth rate. The spectral radius is the better predictor. - *Tie breaking:* the order of the neighbour list changes which branch DFS reaches first, which accounts for its large run-to-run variation in path length between seeds. - *Language overhead:* Python's dictionary and deque operations cost a roughly constant factor per node, so the measured time grows slightly faster than the operation counts at the largest size (cache effects), while the counts themselves match the theory.

TASK 11 – Recommend and Defend the Final Design [4 Marks]

Recommendation: BFS-GRAPH (approach A) with an adjacency list and an explored

set.

Why BFS and not the alternatives. - It is the only approach that returned the optimal route at every size tested, which is what matters when medicines must reach the ward in minimum time. - DFS returns routes up to about 5,000 times longer at 1,000,000 locations, which is unacceptable for time critical deliveries. - IDDFS is optimal and uses little memory, but its time grows as λ^d and it could not finish at 100,000 or 1,000,000 locations within the budget.

Theoretical and experimental efficiency. BFS has O(N + E) time and memory. The experiments confirm it: at 1,000,000 locations it takes 1.6 seconds and uses about 1.2 million stored entries, with predicted and measured counts within 2 percent.

Preprocessing cost and trade-offs. Generating the graph, building its adjacency lists and selecting the goal takes about 7.4 seconds at 1,000,000 locations. The main trade-off is memory: the explored set grows with the hospital, so at much larger networks (about ten times the current size) the frontier and explored set would need a more compact representation or a bidirectional search.

Is it practical as the hospital grows? Yes, for the sizes required (1,000 to 1,000,000 states). Time and memory grow linearly, so doubling the hospital roughly doubles the work. The design would not be practical for networks far beyond this without an informed search, which is outside the uninformed scope of this assignment, or a memory-efficient BFS variant.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Challenge) [8 Marks]

Changed requirement. After a power restart or sensor fault, the robot knows only that it is at one of k possible junctions and must still reach the pharmacy.

A. Impact Analysis [2 Marks]

- **Changed assumptions.** The original design assumed a fully observable environment and a known start state. Both are now false: the robot's position is partly unknown and its percepts are incomplete.
- **Why knowledge becomes part of the state.** The robot cannot plan a route to the pharmacy from a single location, because the correct first move depends on where it actually is. The state must therefore include what the robot believes, a set of possible locations (a belief state). The search must now find one sequence of actions that works for every possible location at once.

B. Evaluation of the Original Design [2 Marks]

- **Can the original algorithm still be applied?** Yes, but the “state” becomes a belief state (a set of locations), and the successor function applies the same action to every location in the set.
- **Size of the new search space.** For k possible locations the number of belief states is at most 2^N, where N is the number of locations. This is the worst case; in the corridor below the reachable beliefs are far fewer because moves merge positions.
- **Is the original design acceptable?** For small k, yes. For large k the exponential growth becomes a limit, so the design must be modified.

C. Adaptation or Redesign [2 Marks]

Decision: MODIFY. The original BFS with an explored set is retained, but its state is changed from a location to a belief state. The local sensor is added as a second option, using an AND-OR search, which branches on the sensor reading (the observation) and produces a conditional plan.

Test model: a corridor of N = 4k + 4 positions with the pharmacy at position 0 and walls at both ends. The robot’s possible start positions are spaced four apart. The actions are left, right and stay. With the local sensor, each junction has a colour (0 or 1), and the robot reads its colour.

Lower bound on plan length: the farthest possible start is 4(k − 1) + 3 positions from the pharmacy, so every plan needs at least 4k − 1 moves. For k = 8 that is 31 moves.

D. Evidence-Based Defence [2 Marks]

Table 12.1: Belief-state search results (seed 7). “Generated” counts belief states generated; “distinct” counts unique belief states.

k	No sensor: beliefs generated	No sensor: distinct beliefs	No sensor: plan length	Sensor: beliefs generated	Sensor: plan depth
2	52	24	7	171	7
4	382	151	15	1,119	15
8	2,941	1,058	31	7,896	31

Findings. - Without a sensor, the belief-state BFS reaches the pharmacy with a plan of 7, 15 and 31 moves for k = 2, 4 and 8. These equal the lower bound, so the search is optimal. The number of distinct belief states grows from 24 to 151 to 1,058, far below the worst case 2^N, because moves merge positions in the corridor. - With the local sensor, the AND-OR search finds a conditional plan of the same depth, and it generates more belief states (171, 1,119 and 7,896) because it explores each colour outcome. - **The sensor does not shorten the plan in this corridor.** Every robot can reach the pharmacy by moving left, and the wall at the pharmacy end makes all positions converge without needing to know where it is. The sensor is useful when junctions diverge and a single action cannot serve all possible positions; this corridor does not have that structure. This is an honest result: the sensor helps less than expected here.

Recommendation for Day 4. Retain the BFS with an explored set, modified to search over belief states. Add the local sensor only where junctions can be told apart by a sensor reading and paths

diverge. The cost is a larger search space: without a sensor, 151 distinct belief states at k = 4 and 1,058 at k = 8, compared with 24 at k = 2.

SUBMISSION CHECKLIST

Requirement	Status
All 12 tasks answered	Yes
Three distinct strategies (iterative BFS, iterative DFS, recursive IDDFS)	Yes
Basic operation, complexity derivations and recurrence solved	Yes
Best, average and worst cases with reasons	Yes

Implementation tested for reachable, unreachable and start = goal cases	Yes (IDDFS unreachable case reported as DNF)
Seeds reported; timings averaged over 5 runs	Yes
Tables and log-scale graphs	Yes
Theory compared with experiment	Yes
Day-4 belief-state experiment for k = 2, 4, 8 with and without sensor	Yes

APPENDIX – Submitted code files

• hospital_search.py – graph generator and the three search algorithms. • test_correctness.py – reachable, unreachable and start = goal tests. • experiment.py – timed experiments for all sizes (writes results.json). • belief_search.py – Day-4 belief-state and AND-OR search (writes the table in Task 12).